

角色设定

你现在是一位顶级的 AI 算法架构师与物流供应链预测专家。你需要帮我梳理一篇题为《A multi-source multi-layer-based transfer learning approach for forecasting customer demands of newly launched products》的论文中的预测实验整体框架。

任务目标

我的最终目的是将你输出的框架发送给高级程序员（Claude），让其使用 Python 开发环境（基于 TensorFlow/Keras 和 scikit-learn 开发，需适配 Mac M1 芯片环境）完整编写出实验代码。因此，你的描述必须具有极强的伪代码感和软件工程结构，做到详尽、准确、毫无遗漏。请不要自己编写最终代码，而是输出详尽的“代码开发蓝图”。

已思考 39s >

下面我给你的不是“论文总结”，而是一份可以直接交给高级程序员去落地的“代码开发蓝图”。我会尽量按软件工程视角，把论文里的预测实验拆成：目标、数据流、模块边界、训练流程、实验矩阵、评价逻辑、以及实现时必须注意的论文歧义点。整体依据论文的实验设计整理。  A multi-source multi-layer-base...  A multi-source multi-layer-base...

先给一个一句话总览。

这篇论文的核心实验框架是：把“新上市产品需求预测”定义为一个目标域时间序列预测任务，先用 KNN 在候选源域商品中找到与目标商品最相近的若干来源，再用 1D CNN 作为统一骨干网络，分别构造 No-TL、单源 TL、三种多源 TL（加权输出、切换式、加权层参数融合），最终提出 MSML-TL-RFE：先做 RFE 特征筛选，再对多个源模型的层参数做加权融合，最后在目标域少量样本上微调，并把预测结果送入两级供应链成本模型，比较 RMSE、准确率与供应链总成本。  A multi-source multi-layer-base...  A multi-source multi-layer-base...

一、你应该交给 Claude 的总开发目标

请 Claude 实现一个“可配置的实验平台”，而不是只写一个模型脚本。这个平台至少应支持：

1. 统一数据接口：支持 3 个数据集，支持 source/target 划分，支持特征抽取与滑窗构造。论文使用 3 个公开数据集，且将数据重组为层级供应链结构。
A multi-source multi-layer-base...
2. 统一相似源检索接口：基于 KNN + Euclidean distance，从候选源中选出 top-k 相似商品/门店。论文明确将 KNN 作为源选择核心。 A multi-source multi-layer-base...
3. 统一骨干模型接口：1D CNN 作为所有方法的基础网络。No-TL、SS-TL、MSWA-TL、MSSB-TL、MSML-TL-RFE 都建立在同一 CNN 主干之上。
A multi-source multi-layer-base...
4. 多种 TL 策略插件化：No-TL、SS-TL、MSWA-TL、MSSB-TL、MSML-TL、MSML-TL-RFE。
5. 统一实验调度器：支持不同 forecasting horizon、不同 source 数量、是否信息共享、重复试验求均值。
6. 统一评估器：RMSE 为主，准确率定义为 $1/\text{RMSE}$ ；并支持供应链成本仿真。论文最终不仅比较 RMSE，还将预测结果带入供应链成本模型。 A multi-source multi-layer-base...
A multi-source multi-layer-base...

二、论文中的实验对象到底是什么

论文不是在做“单纯的时间序列预测”，而是在做“新产品冷启动预测 + 迁移学习 + 供应链决策验证”。

具体假设是：

- 目标产品刚上市约 1 个月。
- 只有这 1 个月左右的目标域数据可用于训练/验证。
- 需要预测未来 6 个月需求，用于后续库存和供应链成本分析。
A multi-source multi-layer-base...
- 源域来自历史上已销售较久的相似产品。
- 数据被重构为一个“两级供应链、3 个零售商 + 1 个供应商”的层级结构。
- 存在两种场景：不共享信息、共享信息。共享信息时，某门店可以从其他门店商品中寻找更相似的源。 A multi-source multi-layer-base...

所以，从代码视角，这个平台的实验单位不是“一个 dataset”，而是：

```
(dataset, scenario, target_item, horizon, method, num_sources, repeat_id)
```

三、数据层应该如何建模

建议 Claude 用下面的数据抽象。

1. 原始数据层 RawRecord

字段至少包括：

- date
- entity_id (store_id / brand_id / product_group_id)
- item_id 或 product_id
- sales
- 以及额外特征：promo、holiday、store_open、customer_count 等，按数据集实际情况决定。论文指出 Dataset 1 只有基础销售信息，Dataset 2 有促销信息，Dataset 3 还有门店相关特征、节假日、门店开闭、客户信息等。  A multi-source multi-layer-base...

2. 目标任务层 TaskSpec

- dataset_name
- target_entity
- target_item
- scenario: no_info_sharing / with_info_sharing
- source_pool_scope
- forecast_horizon_days
- lookback_window = 10
- source_top_k = 3 (默认)
- repeat_times

3. 时序样本层 WindowSample

- X: shape = (window_size, num_features)
- y: scalar
- meta: date_end, source_id, target_id, horizon

论文明确说明：

- 先做 min-max 归一化。
- 再把数据按时序组织成 2D 表。
- 然后用 sliding window 转成 CNN 所需 3D 张量。
- 使用窗口大小 10。

四、数据预处理蓝图

Claude 应把预处理拆成 6 个明确函数。

1. load_dataset(config) -> pd.DataFrame

按数据集名加载 CSV，并做基础字段对齐。

2. `extract_datetime_features(df) -> df`

论文显式抽取：

- year
- month
- week
- day

并拼接其他已有特征。  A multi-source multi-layer-base...

3. `normalize_features(train_df, val_df, test_df, fit_on="train")`

使用 min-max scaling, 把特征缩放到 [0,1]。论文给出了 min-max 公式。

 A multi-source multi-layer-base...

4. `build_source_target_split(df, task_spec)`

论文对 source 和 target 的划分不是随机的，而是按“长期商品 vs 新上市商品”设定。对于 Dataset 1, 前 9 个 item 可作为 source, 第 10 个 item 作为 target。source 用较长历史, target 只拿约 1 个月做 train/val, 未来 6 个月做 test。

5. `build_tabular_sequence(df, horizon, window_size=10)`

将结构化特征转为监督学习格式：

- X_t = 过去 10 个时间步的多特征矩阵
- y_t = 第 $t + \text{horizon}$ 天销量

论文文字有一处不完全一致：一处写中期 $\text{horizon}=15$, 另一处写窗口 10 可做 5 天 ahead, 且最终因目标域数据少, 仅做到 5 天 ahead。实现时必须参数化, 不要把 horizon 写死。默认支持 horizon in [1,2,3,4,5,15,30], 但主实验优先实现 1~5。

6. `to_cnn_tensor(X_2d) -> X_3d`

shape 从 (n_samples, window_size, num_features) 保持为 3D, 直接供 Keras Conv1D 使用。

五、训练/验证/测试划分怎么做

论文的关键是 source 和 target 划分规则不同。

Source domain:

- 大约 80% train
- 约 10% val
- 约 10% test

Target domain:

- 训练与验证都很少, 各约 6%–7%
- 测试约 85%–86%

- Dataset 1 的示例是：target train 15 天，val 15 天，test 6 个月。

📄 A multi-source multi-layer-base...

因此代码不能写成普通随机切分，必须实现：

```
temporal_split_by_ratio_or_dates()
```

并支持两种模式：

- 论文日期复现模式
- 通用比例模式

六、源选择模块是整个系统的前置核心

这个模块的职责不是直接训练模型，而是为 TL 筛选最相似的源任务。

建议 Claude 实现：

```
SourceSelector.fit(candidate_source_meta_features)
SourceSelector.get_top_k_sources(target_signature, k=3, scenario=...)
```

关键逻辑：

1. 对目标产品的“上市首月模式”构造特征签名。
2. 在候选源池中检索 top-k 最相近源。
3. 使用 KNN + Euclidean distance。

这里有一个非常重要的论文歧义，必须告诉程序员：

论文正文说，多源加权时使用的是 inverse weighted distance，也就是距离越小权重越大。

但论文伪代码 Algorithm 1 中写成了：

```
weights_avg = [distance_i / sum_distance]
```

这会导致距离越大权重越大，逻辑上与“相似源应更高权重”相反。

因此建议实现为：

- weight_mode="inverse_distance" 作为默认论文忠实解释
- 额外保留 weight_mode="raw_distance" 以便复现实验伪代码版本
- 输出实验日志时明确标注当前使用哪一种

推荐默认公式：

$$w_i = (1 / (d_i + \text{eps})) / \sum_j (1 / (d_j + \text{eps}))$$

七、CNN 主干网络应该如何抽象

论文的骨干结构很明确：

- 3 个 Conv1D
- Conv/Conv 之间有 MaxPooling
- 后接 Flatten
- 后接 Dense
- 在 TL 中，前 4 层冻结，其余低层/后层可训练。  A multi-source multi-layer-base...

给 Claude 的实现蓝图应是：

```
build_base_cnn(input_shape, output_dim=1, config) -> tf.keras.Model
```

建议层级抽象成：

- conv1
- pool1
- conv2
- pool2
- conv3
- flatten
- dense_out

并让每层有稳定名字，便于冻结和跨模型参数融合，例如：

- conv1
- pool1
- conv2
- pool2
- conv3
- flatten
- dense

冻结策略默认：

- freeze first 4 layers
- keep later layers trainable

八、五种预测方法的代码级实现蓝图

1. No-TL

定义：只在目标域少量数据上训练同一个 CNN，不使用任何源域知识。

 A multi-source multi-layer-base...

流程：

- 构造 target train/val/test
- 训练 base CNN
- 在 target test 上预测
- 输出 RMSE

伪代码：

```
target_data = prepare_target_data(task)
model = build_base_cnn(...)
train(model, target_train, target_val)
pred = model.predict(target_test)
rmse = compute_rmse(pred, y_test)
```

2. SS-TL

定义：选 1 个最相似源，先在源上预训练，再把前 4 层冻结迁移到目标域微调。

 A multi-source multi-layer-base...

流程：

- KNN 找到最相似单源 S1
- 在 S1 上训练 source model
- 新建 target model，复制 source model 前 4 层参数并冻结
- 在 target_train/val 上微调
- test 评估

伪代码：

```
source = select_top_k(..., k=1)[0]
source_model = train_source_cnn(source)
target_model = build_base_cnn(...)
transfer_and_freeze_first_n_layers(source_model, target_model, n=4)
fine_tune(target_model, target_train, target_val)
pred = predict(target_model, target_test)
```

3. MSWA-TL

定义：分别对多个源独立做 SS-TL，每个模型各自产生预测，然后按源权重对“预测值”做加权求和。  A multi-source multi-layer-base...

注意：

这里融合的是输出，不是层参数。

伪代码：

```
sources, distances = select_top_k(..., k=3)
weights = compute_source_weights(distances)

preds = []
for source in sources:
    model_i = train_ss_tl(source, target_data)
    pred_i = model_i.predict(target_test)
    preds.append(pred_i)

final_pred = weighted_sum(preds, weights)
rmse = compute_rmse(final_pred, y_test)
```

4. MSSB-TL

定义：多个源各自形成一个 SS-TL 模型，但最终不做加权平均，而是在 target validation 上选择验证 RMSE 最低的那个模型，并用它做测试预测。

 A multi-source multi-layer-base...

伪代码：

```
best_model = None
best_val_rmse = inf

for source in sources:
    model_i = train_ss_tl(source, target_data)
    val_pred_i = model_i.predict(target_val)
    val_rmse_i = rmse(val_pred_i, y_val)
    if val_rmse_i < best_val_rmse:
        best_val_rmse = val_rmse_i
        best_model = model_i

test_pred = best_model.predict(target_test)
test_rmse = rmse(test_pred, y_test)
```

5. MSML-TL

定义：不是对输出加权，而是对多个源模型的“对应层参数”做加权融合，生成一个新的 target model 初始冻结层，再在目标域微调。  A multi-source multi-layer-base...

关键：

- 多个源各自训练 source CNN
- 对对应层 weights/biases 按源权重做加权平均
- 把这个 averaged layer weights 赋给新的 target CNN 的冻结层

- 其余层可训练
- 再在目标域 fine-tune

伪代码：

```
source_models = [train_source_cnn(s) for s in sources]
weights = compute_source_weights(distances)

fused_params = {}
for layer_name in transferable_layers:
    layer_params = [m.get_layer(layer_name).get_weights() for m in source_models]
    fused_params[layer_name] = weighted_average_params(layer_params, weights)

target_model = build_base_cnn(...)
load_fused_params_into_layers(target_model, fused_params)
freeze_layers(target_model, transferable_layers)
fine_tune(target_model, target_train, target_val)
```

6. MSML-TL-RFE

定义：在 MSML-TL 基础上加入 RFE，对 target 和 selected sources 先做特征筛选，再训练、融合、微调。论文说每个数据集大约保留 40%–60% 特征。

 A multi-source multi-layer-base...

伪代码：

```
sources, distances = select_top_k(..., k=3)

selected_features = run_rfe_on_target_and_sources(target_df, source_dfs)
target_df_rfe = target_df[selected_features]
source_dfs_rfe = [df[selected_features] for df in source_dfs]

target_windows = build_windows(target_df_rfe)
source_windows = [build_windows(df) for df in source_dfs_rfe]

source_models = [train_source_cnn(sw) for sw in source_windows]
fused_params = fuse_layerwise_params(source_models, distances, mode="inverse_dist")

target_model = build_base_cnn(input_shape=...)
set_fused_frozen_layers(target_model, fused_params)
fine_tune(target_model, target_train, target_val)
test_pred = target_model.predict(target_test)
test_rmse = rmse(test_pred, y_test)
```

九、RFE 模块应如何实现

论文没有把 RFE 的底层 estimator 说得非常细，因此实现上要做成可配置。

建议接口：

```
run_rfe(X, y, estimator_name="random_forest", keep_ratio=0.5)
```

推荐默认：

- estimator 用 RandomForestRegressor 或 LinearRegression
- keep_ratio 默认 0.5
- 支持搜索 [0.4, 0.5, 0.6]

因为论文说保留大约 40%–60% 特征，而不是单一比例。 [A multi-source multi-layer-base...](#)

重要实现原则：

- RFE 必须只在训练阶段拟合，不能泄漏 test 信息。
- 如果 source 与 target 的原始特征并不完全一致，需要先做特征对齐，再做 RFE。
- 最终 CNN 的输入维度由筛选后的特征数决定。

十、实验调度器应该怎么设计

论文是“同一实验多次重复再取均值”。

因此 Claude 不应只写一个 main.py，而应写一个 ExperimentRunner。

推荐核心循环：

```
for dataset in datasets:
    for scenario in ["no_info_sharing", "with_info_sharing"]:
        for horizon in horizons:
            for method in methods:
                for num_sources in source_counts:
                    for repeat in range(n_repeats):
                        run_single_experiment(...)
                        aggregate_mean_std(...)
```

其中：

- methods = [NoTL, SSTL, MSWATL, MSSBTL, MSMLTLRFE]
- source_counts 至少支持 1、3；论文后面也扩展到 6、9。
[A multi-source multi-layer-base...](#)
- horizons 至少支持 1~5；如需严格论文扩展，可加入 15、30，但主实验应先确保 1~5 正确。 [A multi-source multi-layer-base...](#)

十一、信息共享场景如何落地

论文的“信息共享”不是模型结构变化，而是 source pool 范围变化。

📄 A multi-source multi-layer-base...

所以代码里应实现：

- no_info_sharing :
source 只能来自同一 retailer / 同一局部池
- with_info_sharing :
source 可来自多个 retailer / 更大候选池

也就是说，变化点只在：

`get_candidate_source_pool(task_spec)`

而不是 TL 模型本身。

十二、评价指标如何实现

基础预测评价：

- RMSE
- $\text{accuracy} = 1 / \text{RMSE}$

论文摘要直接用 $\text{accuracy} = \text{RMSE}$ 的倒数来表达性能提升。

📄 A multi-source multi-layer-base...

建议同时记录：

- RMSE mean/std
- MAE
- MAPE (可选)
- training_time
- inference_time
- selected_sources
- selected_features_count

十三、供应链成本仿真模块必须独立实现

论文后半部分不是可有可无，它是验证“预测提升是否真的降低供应链成本”的关键。

应实现类：

`TwoEchelonSupplyChainSimulator`

输入：

- predicted_demand_series
- lead_time_mode: static / dynamic
- initial_inventory = 250
- lot_size_Q = 200
- ordering_cost = 20
- purchasing_cost = 2
- holding_cost = 7
- lost_sales_cost = 5
- service_level_z
- lead_time_series or fixed_lead_time

论文给出的核心公式：

1. 安全库存

$$ss = Z * \sqrt{(\sigma_d^2) * L_{\text{bar}} + (\sigma_L^2) * d_{\text{bar}}}$$

若固定 lead time, 则

$$ss = Z * \sigma_d * \sqrt{L}$$

 A multi-source multi-layer-base...

2. 再订货点/订货量阈值

$$RA = LTPD + ss$$

其中 LTPD 是 lead time predicted demand。  A multi-source multi-layer-base...

3. 总成本

$$TC = PC + OC + HC + LS$$

 A multi-source multi-layer-base...

仿真逻辑：

- 逐日滚动
- 更新库存
- 若真实需求 > 可用库存, 则产生 lost sales
- 若 $RA > \text{inventory_on_hand} + \text{inventory_in_transit}$, 下单补货
- 订单按 lead time 到货
- 累积 4 类成本

十四、建议的工程目录结构

你可以直接把下面这个结构发给 Claude：

```
project/
├─ configs/
│   ├─ datasets.yaml
│   ├─ model.yaml
│   └─ experiments.yaml
```

```

├── supply_chain.yaml
├── data/
├── src/
│   ├── data/
│   │   ├── loaders.py
│   │   ├── feature_engineering.py
│   │   ├── normalization.py
│   │   ├── splitters.py
│   │   └── windowing.py
│   ├── source_selection/
│   │   ├── knn_selector.py
│   │   └── distance_weighting.py
│   ├── models/
│   │   ├── cnn_backbone.py
│   │   ├── transfer_utils.py
│   │   └── layer_fusion.py
│   ├── feature_selection/
│   │   └── rfe_selector.py
│   ├── methods/
│   │   ├── no_tl.py
│   │   ├── ss_tl.py
│   │   ├── mswa_tl.py
│   │   ├── mssb_tl.py
│   │   ├── msml_tl.py
│   │   └── msml_tl_rfe.py
│   ├── evaluation/
│   │   ├── metrics.py
│   │   ├── aggregators.py
│   │   └── reporting.py
│   ├── supply_chain/
│   │   └── simulator.py
│   ├── experiments/
│   │   ├── runner.py
│   │   └── task_builder.py
│   └── utils/
│       ├── seed.py
│       ├── logging.py
│       └── io.py
├── notebooks/
└── .

```

<> Plain text



十五、Mac M1 / TensorFlow-Keras / scikit-learn 适配要求

由于你已经明确需要适配 Mac M1，建议把下面这些工程要求也一并交给 Claude：

1. 使用 tensorflow-macos 和 tensorflow-metal 路线。
2. 所有 numpy / tensorflow / python random 都要设 seed。
3. 尽量避免自定义算子，使用 Keras 原生层。
4. KNN、RFE、MinMaxScaler 使用 scikit-learn。

5. 不要依赖 GPU 专属 CUDA 逻辑。

6. 训练并行应谨慎；论文自己也承认顺序训练耗时较高，未来可并行化。实现第一版先保证正确性，再加并行。  A multi-source multi-layer-base...

十六、必须向 Claude 强调的论文歧义与复现风险

这是最重要的一段，因为如果不说明，程序员很容易“写出能跑的东西”，但不是真正接近论文实验。

第一，权重公式存在矛盾。正文强调 inverse distance weighting，但伪代码写成 raw distance normalization。实现时必须双版本支持，并默认使用 inverse-distance，因为它更符合相似源加权逻辑。

第二，forecast horizon 的描述存在不完全一致。论文一处提到 1、15、30 天，一处又说窗口 10 用于 1-day 和 5-day，且最后因为 target data 太少只做 5 天 ahead。实现时必须参数化 horizon，主复现实验优先 1~5。

第三，RFE 的具体 estimator 没有写死。需要做成配置项，不能硬编码成唯一方法。

 A multi-source multi-layer-base...

第四，CNN 的详细超参数在你当前给我的截取内容里没有完整列出，所以第一版应把 filters、kernel_size、activation、optimizer、epochs、batch_size 全部做成配置文件项，而不是在代码中写死。我目前能确认的是网络拓扑结构与冻结策略，但不能据此断言具体超参数值。  A multi-source multi-layer-base...

第五，源选择“至少 3 个”是主方案，但论文后期还扩展到 6 和 9 个源。第一阶段先复现 3-source MSML-TL-RFE，第二阶段再扩展 source_count ablation。

 A multi-source multi-layer-base...

十七、可以直接发给 Claude 的“开发指令版摘要”

你可以把下面这段原样发给 Claude：

请实现一个可复现实验平台，用于复现论文《A multi-source multi-layer-based transfer learning approach for forecasting customer demands of newly launched products》的预测实验。核心流程为：对数据做日期特征抽取与归一化，构建 source/target domain，使用滑窗将多变量时序转为 3D CNN 输入；基于目标产品首月数据，用 KNN + Euclidean distance 从候选源池中检索 top-k 相似源；统一以 1D CNN 为骨干实现 No-TL、SS-TL、MSWA-TL、MSSB-TL、MSML-TL、MSML-TL-RFE；其中 MSML-TL-RFE 需先做 RFE，再对多个源模型对应层参数做加权融合，并冻结迁移层后在目标域少量数据上微调；实验需支持信息共享与非共享两种 source pool；需支持多次重复实验并输出均值

RMSE; 准确率定义为 $1/\text{RMSE}$; 预测结果还需送入两级供应链成本仿真模块, 计算 PC、OC、HC、LS 与总成本。注意论文对 source weighting 存在歧义: 正文强调 inverse distance weighting, 但伪代码使用 raw distance normalization, 请实现双模式并默认 inverse-distance。

十八、我的结论

如果你要把这篇论文真正“翻译成工程实现”, 最核心的不是 CNN 本身, 而是四层实验逻辑必须完整保留:

- 数据被重构成 source/target 的冷启动问题;
- 相似源选择是 TL 成败前置条件;
- 多源知识迁移有三种不同融合位点: 输出层、模型选择层、参数层;
- 最终验证目标不是只有预测误差, 而是供应链成本。

如果你需要, 我下一步可以继续把这份蓝图进一步整理成“给 Claude 的逐文件开发清单”, 也就是精确到每个 .py 文件该写哪些类、哪些函数、输入输出签名是什么。

来源

